

Project Report

AI Discord Tutor

A) Introduction

Continuous learning is widely recognized as one of the most valuable habits a student can build, yet keeping it up day after day is far from straightforward. Beyond the time it takes to identify what to study, the discipline of coming back to a subject regularly is genuinely hard to sustain, and that difficulty is rarely about a lack of interest in the material itself. It comes from a structural gap outside formal coursework: there is no system that pushes a learner to engage with a topic again, no single place that gathers the right resources, and no entity that adapts the pace of revision to the individual.

This gap shows up in three concrete ways. First, regularity and stimulation are missing, since students have no guided learning path once they step outside their courses. Second, resources are fragmented, as relevant articles and videos are scattered across the web and simply finding them takes time. Third, learning stays impersonal, because a single static resource cannot account for the fact that everyone learns differently.

This project addresses that gap by reversing the usual model: instead of relying on the student to remember to revise, the tutor itself reaches out, on a platform they already use every day, Discord, with the right content delivered automatically and on time.

Continuous learning is widely recognized as one of the most valuable habits a student can build, yet keeping it up day after day is far from straightforward. Beyond the time it takes to identify what to study, the discipline of coming back to a subject regularly is genuinely hard to sustain, and that difficulty is rarely about a lack of interest in the material itself. It comes from a structural gap outside formal coursework: there is no system that pushes a learner to engage with a topic again, no single place that gathers the right resources, and no entity that adapts the pace of revision to the individual.

This gap shows up in three concrete ways. First, regularity and stimulation are missing, since students have no guided learning path once they step outside their courses. Second, resources are fragmented, as relevant articles and videos are scattered across the web and simply finding them takes time. Third, learning stays impersonal, because a single static resource cannot account for the fact that everyone learns differently.

This project addresses that gap by reversing the usual model: instead of relying on the student to remember to revise, the tutor itself reaches out, on a platform they already use every day, Discord, with the right content delivered automatically and on time.

B) Goal

The goal of this project is to build an AI tutor that takes initiative in the learning process rather than waiting passively to be asked. Three concrete capabilities define this objective:

- On-demand revision: the user can directly ask the agent to start a revision session whenever they want.
- Daily autonomous delivery: the tutor sends a message every day on its own, without needing a new request each time.
- Document-based revision: the user can supply their own documents (lecture notes, PDFs) and have the tutor build lessons strictly from that material.

Around these three points, the system was designed with two complementary modes: a Revision mode, grounded only in the documents the user provides, and a Discovery mode, which lets the agent introduce new subjects using information found online when no document is supplied.

C) Workflow

To support both spontaneous requests and fully autonomous delivery, the agent is organized around two complementary workflows: planning a lesson and sending a lesson.



Figure 1: Workflow

Planning a lesson starts when the user sends a direct message to the AI on Discord. Once the agent is activated, it determines which of the two modes the request falls into. In Revision mode, an MCP tool fetches and processes the document supplied by the user, and the conversation is restricted to that content only. In Discovery mode, the agent instead calls an MCP tool to retrieve information from the internet, in order to introduce a topic the student has not studied before. In both cases, the retrieved material is used to generate a syllabus, after which a second MCP tool call manages the agent's memory, recording the subject, the multi-day plan, and the schedule on which lessons should later be sent. This planning phase is what allows the tutor to commit to a learning path that spans several days rather than a single exchange.

Sending a lesson, by contrast, is triggered independently by a cron-like scheduler that fires according to the plan and the recipient's time zone. Each trigger activates the agent in a stateless way: rather than relying on an ongoing conversation, it fetches the relevant subject, context, and resources directly from what was stored during planning. The agent then composes the lesson and sends it as a direct message. Separating planning, which happens once with user input, from sending, which repeats automatically without it, is what makes the tutor proactive rather than purely reactive.

D) MCP Tools

Each step of these two workflows is carried out by the LLM agent through a set of MCP (Model Context Protocol) tools, which act as its hands on the external world. 6 tools support the system:

- PDF Extractor: extracts and chunks PDF documents into structured content, used in Revision mode to ground lessons strictly in the material the student supplied.
- YouTube Scraper: finds pedagogically relevant videos matched to the topic and the learner's level, used in Discovery mode to enrich lessons with external resources.
- Track Manager: handles the agent's memory by building and updating the multi-day learning path, storing the subject, the plan, and the delivery schedule between the planning and sending phases.
- Cron: schedules and triggers the autonomous delivery of daily lessons based on the user's time zone, decoupling lesson generation from any direct user action.
- Lesson formater: formats the lesson generated by the LLM into a Discord-ready message, structuring its content, length, and layout so that it matches Discord's display constraints. It does not deliver the message itself, only prepares it, which removes the need to manually adjust formatting for every lesson and keeps the output consistent regardless of the subject.
- Discord: delivers the formatted lesson as a direct message to the student's inbox, and is also the entry point through which the student first talks to the agent.

E) Limitations

Despite reaching its core goal, an autonomous daily tutor, the current version of the system has several notable limitations.

The interaction remains essentially one-directional: the agent generates and sends lessons, but the student has no built-in way to ask a follow-up question or to be quizzed on what was just covered. Without any form of assessment, the tutor also has no way of verifying whether a lesson was actually understood.

Personalization is currently limited to choosing a subject and a delivery time. The system does not adapt its pace, depth, or teaching method to how an individual learner actually progresses, and it does not track which lessons have already been completed versus simply sent.

The output format is restricted to text delivered through Discord DMs, which does not suit learners who would prefer to listen to a lesson, for instance during a commute. Cost and scalability are also a concern, since every lesson generation involves an LLM call together with several MCP tool calls; prompts and token usage have not yet been optimized, which would become a real constraint if the number of learners or daily lessons grew significantly.

Finally, because the sending workflow runs statelessly, the quality of a lesson depends entirely on what was stored during planning. Any gap or inaccuracy introduced at that stage, particularly through the internet-based Discovery mode, is never re-verified afterward and can propagate into the lessons sent later.

F) Conclusion

This project set out to address a concrete frustration: learning regularly takes effort, resources are scattered, and nothing exists to remind a student why and how to keep going. The AI Discord Tutor responds to that gap with an agent that is proactive rather than reactive, structured rather than improvised, and reachable directly inside a platform students already use. By separating the planning of a learning path from the day-to-day delivery of lessons, and by relying on a set of MCP tools to fetch documents or web resources, manage memory and scheduling, and finally communicate with the learner, the system turns a single request into a genuine multi-day follow-up.

G) Perspective

This first version remains an early iteration, and several directions could take it further. On interactivity, the tutor could become bidirectional, letting students ask questions directly in their DMs, while interactive quizzes would turn passive reading into active checks. On personalization, the system could be extended to genuinely adapt to each learner's pace and method, and to track lesson completion and progress over time. On scaling, lessons could be converted into short podcasts for learners who prefer audio, while prompts and token usage would need to be optimized to keep the system cost-effective as the number of users grows.